



IE3010

Network Programming

3rd Year, Semester II

Assignment

NetMessenger: A Multi-Client Chat and File-Sharing Platform over TCP/IP

W.W.D.M.Fernando

IT23629844

Submitted to

Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number: IT23629844

Name: W.W.D.M.Fernando

Executive Summary

NetMessenger is a TCP client/server chat and file-sharing platform implemented in C using the standard BSD sockets API. The application runs on the personalized TCP port 15844 and supports multiple simultaneous users. The implemented system provides user registration, connected-user listing, broadcast messaging, private messaging, presence notifications, chat rooms, direct file transfer, server-side file storage, graceful client disconnection, reconnection, error handling, and timestamped server logging. The completed application was tested using multiple simultaneous clients, including six connected clients, and the main communication, messaging, chat-room, file-transfer, storage, logging, and disconnect functionalities were verified during execution.

This report presents the personalized configuration, system architecture, concurrency approach, communication protocol implementation, functional features, file-sharing mechanism, error handling, server-side logging, source-code evidence, execution and testing evidence, design rationale, development process, and final testing results. Screenshots included in the report are taken from the actual project development and execution environment and are accompanied by appropriate figure titles and explanations.

GIT URL: <https://github.com/DMFernando0315/IT23629844-NetMessenger>

Table of Contents

Executive Summary	3
1.Introduction and Objectives	5
2.Personalization Requirements	6
2.1 Personalization Calculation.....	7
2.2 Project Files.....	7
3. System Overview and Architecture	9
4. Concurrency Model.....	10
5. Communication Protocol Implementation	10
6. Functional Implementation	12
6.1 Registration and Presence	12
6.2 Broadcast Messaging	13
6.3 Private Messaging	14
6.4 Chat Rooms.....	14
7. File Sharing Implementation.....	15
8. Error Handling and Graceful Disconnect.....	16
9. Server-Side Logging	17
10. Annotated Code Evidence.....	18
Socket creation, bind, listen and accept	18
Concurrency/client handling	18
REGISTER and user state.....	18
BCAST and PMSG	19
JOIN, LEAVE, ROOMS and RMSG.....	19
SENDFILE receive/store/forward.....	21
Validation and ERR response.	21
Disconnect cleanup and logging	22
11. Execution and Output Evidence.....	22
12. Testing Summary	28
13. Design Rationale and Assumptions	29
14. Conclusion.....	29
15. Prompt Log / AI Interaction Record	30
16. Structured Reflection	31
17. References	32
18. Appendix	32
Appendix A – Project Files	32

Appendix B – GitHub Repository	32
Appendix C – Evidence Checklist	33

1.Introduction and Objectives

The objective of **NetMessenger** is to demonstrate socket-level TCP network programming in C through a central server that coordinates multiple interactive clients. The application was designed to implement the required communication protocol while handling TCP-based client-server communication and multiple simultaneous connections.

The final implementation supports user registration, connected-user listing, broadcast messaging, private messaging, presence notifications, chat rooms, room messaging, file transfer, graceful termination, error handling, server-side file storage, timestamped event logging, and client reconnection after disconnection.

The main objectives of the project are:

- To use the standard **BSD sockets API** directly in C on Linux.
- To support at least five simultaneous clients from a single server process.
- To implement the required protocol commands, including:
 - REGISTER
 - LIST
 - BCAST
 - PMSG
 - JOIN
 - LEAVE
 - ROOMS
 - RMSG
 - SENDFILE
 - QUIT

- To provide personalized server responses containing NID:6298.
- To implement newline-based command processing for the text communication protocol.
- To implement file transfer using the required file-size information.
- To provide personalized server logging and storage based on registration number IT23629844.
- To demonstrate the functionality through actual execution and testing with multiple clients.
- To provide evidence of messaging, room management, file transfer, logging, error handling and graceful disconnection.

2. Personalization Requirements

All registration-specific values were derived from the registration number **IT23629844** and were used consistently throughout the NetMessenger project.

Item	Final Value	Derivation / Purpose
Registration Number	IT23629844	Full student registration number
Numeric Part	23629844	Numeric portion used for personalization
Last Four Digits	9844	Used to calculate the server port and personalized filenames
Server Port	15844	$6000 + 9844$
Node ID	NID:6298	Digits 3–6 of 23629844 are 6298
Server Source	server_9844.c	Personalized server source filename
Client Source	client_9844.c	Personalized client source filename
Makefile	Makefile_9844	Personalized build file

Server Log	netmsg_IT23629844.log	Personalized server log filename
Storage Root	./storage/IT23629844	Personalized storage directory
Storage Structure	./storage/IT23629844/<sender_username> /<filename>	Files are stored under sender-specific directories

2.1 Personalization Calculation

The server port was calculated using the last four digits of the registration number:

Registration Number = IT23629844

Last Four Digits = 9844

Server Port = 6000 + 9844

= 15844

The Node ID was calculated from digits 3–6 of the numeric portion:

Numeric Part = 23629844

Digits 3–6 = 6298

Node ID = NID:6298

Therefore, the application uses:

TCP Port : 15844

Node ID : NID:6298

2.2 Project Files

The personalized implementation contains:

server_9844.c

client_9844.c

Makefile_9844

README.md

netmsg_IT23629844.log

storage/IT23629844/

```
dulara@10:/media/sf_IT23629844/NetMessenger
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:~$ cd /media/sf_IT23629844/NetMessenger
dulara@10:/media/sf_IT23629844/NetMessenger$ printf "Registration Number : IT23
629844\nNumeric Part      : 23629844\nLast Four Digits    : 9844\nServer Port
15844\nNode ID        : NID:6298\nServer Source      : server_9844.c\nClient S
ource      : client_9844.c\nMakefile      : Makefile_9844\nLog file      : netms
g_IT23629844.log\nStorage Root      : ./storage/IT23629844\n\nProject Files:\n"
Registration Number : IT23629844
Numeric Part      : 23629844
Last Four Digits    : 9844
Server Port      15844
Node ID        : NID:6298
Server Source    : server_9844.c
Client Source    : client_9844.c
Makefile        : Makefile_9844
Log file        : netmsg_IT23629844.log
Storage Root     : ./storage/IT23629844

Project Files:
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Figure 1: Configuration values

The final NetMessenger server is shown listening on the personalized TCP port **15844** on the IPv4 wildcard address 0.0.0.0. The output demonstrates that the server is actively listening for TCP client connections, while the connected client provides evidence that the server is being accessed through the configured port.

```
dulara@10:/media/sf_IT23629844/NetMessenger
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:~$ cd /media/sf_IT23629844/NetMessenger
dulara@10:/media/sf_IT23629844/NetMessenger$ printf "Registration Number : IT23
629844\nNumeric Part      : 23629844\nLast Four Digits    : 9844\nServer Port
15844\nNode ID        : NID:6298\nServer Source      : server_9844.c\nClient S
ource      : client_9844.c\nMakefile      : Makefile_9844\nLog file      : netms
g_IT23629844.log\nStorage Root      : ./storage/IT23629844\n\nProject Files:\n"
Registration Number : IT23629844
Numeric Part      : 23629844
Last Four Digits    : 9844
Server Port      15844
Node ID        : NID:6298
Server Source    : server_9844.c
Client Source    : client_9844.c
Makefile        : Makefile_9844
Log file        : netmsg_IT23629844.log
Storage Root     : ./storage/IT23629844

Project Files:
dulara@10:/media/sf_IT23629844/NetMessenger$ ss -tlnp | grep 15844
LISTEN 0      50          0.0.0.0:*    0.0.0.0:*   users:((("server_9844"
,pid=6144,fd=3))
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Figure 2: TCP Port 15844 LISTEN

The final NetMessenger server is shown listening on 0.0.0.0:15844. The TCP connection information also shows client connection states associated with the server port, providing evidence that the personalized listener is active and has been used by a TCP client.

3. System Overview and Architecture

NetMessenger follows a client-server architecture. A single C server listens on TCP port 15844 and accepts connections from multiple C clients. Each client registers a unique username before using the main application commands.

The server maintains connected-user and room state and coordinates messaging and file-transfer operations. Broadcast messages are delivered to other registered clients, private messages are delivered only to the named target, and room messages are delivered to room members.

During file sharing, the server receives the declared file data, stores a server-side copy under the personalized storage path and forwards the file to the requested destination.

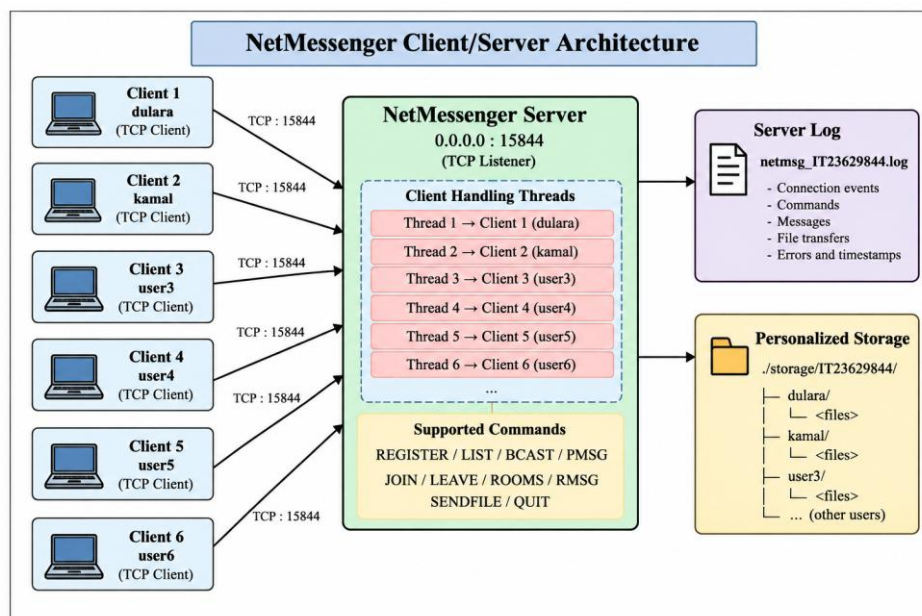
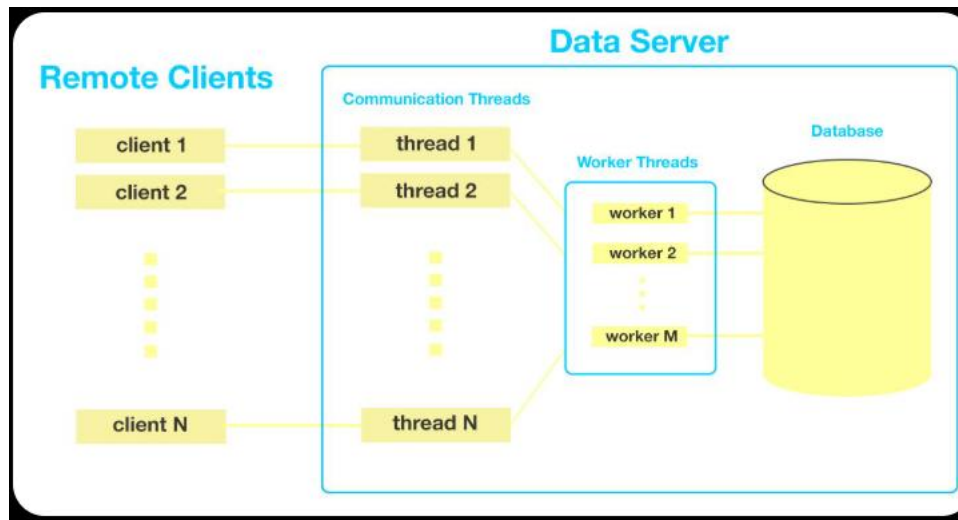


Figure 3: NetMessenger client/server architecture

Multiple C clients communicate with the central server through TCP. The server listens on port 15844, manages users and rooms, routes messages, handles file transfers, records events and stores transferred files.



4. Concurrency Model

The submitted implementation uses a thread-per-client concurrency model. When the server accepts a TCP connection, the connection is handled independently so that multiple clients can communicate with the server at the same time.

POSIX threads are suitable for the bounded coursework workload because each client can be processed independently while the server continues accepting and serving other clients. The model is straightforward to implement and avoids forcing all client operations through one sequential connection handler.

The implementation was tested using six simultaneous clients: dulara, kamal, user3, user4, user5 and user6. This exceeds the assignment requirement of at least five simultaneous clients.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -E "accept\(|pthread_create"
" server_9844.c
564:     int fd = accept(server_fd, (struct sockaddr *)&cliaddr, &len);
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Figure 4: Server connection acceptance and per-client handling

The server accepts an incoming TCP connection and creates independent handling for the connected client, allowing multiple clients to remain active concurrently.

5. Communication Protocol Implementation

The implementation follows the fixed command set specified by the assignment. Text commands and responses use newline termination. SENDFILE is handled specially because the command header is followed by the declared raw file data.

Server-generated OK and ERR responses use the personalized Node ID tag NID:6298. Forwarded message notifications are separate from acknowledgement responses.

Command	Purpose	Successful behaviour
REGISTER <username>	Register unique user	OK REGISTERED <username> NID:6298
LIST	List connected users	OK USERS ... NID:6298
BCAST <message>	Broadcast	OK SENT; other clients receive MSG BCAST
PMSG <username> <message>	Private message	OK SENT; target receives private message
JOIN <room>	Create/join room	OK JOINED <room> NID:6298
LEAVE <room>	Leave room	OK LEFT <room> NID:6298
ROOMS	List rooms	OK ROOMS ... NID:6298
RMSG <room> <message>	Room message	OK SENT; room members receive message
SENDFILE <target> <filename> <filesize>	File transfer	File transferred, stored and acknowledged
QUIT	Graceful disconnect	OK BYE NID:6298

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/media/sf_IT23629844/NetMessenger$ cd /media/sf_IT23629844/NetMessenger
dulara@10:/media/sf_IT23629844/NetMessenger$ ./client_9844 127.0.0.1 kamal
OK REGISTERED kamal NID:6298
Commands:
LIST
BCAST <message>
PMSG <username> <message>
JOIN <room>
LEAVE <room>
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
>
```

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/media/sf_IT23629844/NetMessenger$ QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone

OK SENT NID:6298
> > PMSG kamal Hello Kamal

OK SENT NID:6298
> > █
```

6. Functional Implementation

6.1 Registration and Presence

REGISTER is used to register a unique username on a new connection. The server stores the username and makes the user available to the other connected clients. Presence notifications are generated when users join or leave.

During testing, dulara registered successfully and additional users including kamal, user3, user4, user5 and user6 were connected. Presence JOIN notifications were observed.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi | dulara@10:/medi | dulara@10:/n x | dulara@10:/medi | dulara@10:/medi

OK REGISTERED kamal NID:6298
Commands:
  LIST
  BCAST <message>
  PMSG <username> <message>
  JOIN <room>
  LEAVE <room>
  ROOMS
  RMSG <room> <message>
  SENDFILE <target> <filename> <local-path>
  QUIT
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
>
MSG BCAST dulara Hello everyone
>
MSG PRIV dulara Hello Kamal
> █
```

6.2 Broadcast Messaging

BCAST sends a message from one registered client to the other connected clients. The sender receives an acknowledgement while other clients receive the broadcast notification.

Broadcast messaging was tested using multiple connected clients and the message was successfully delivered.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

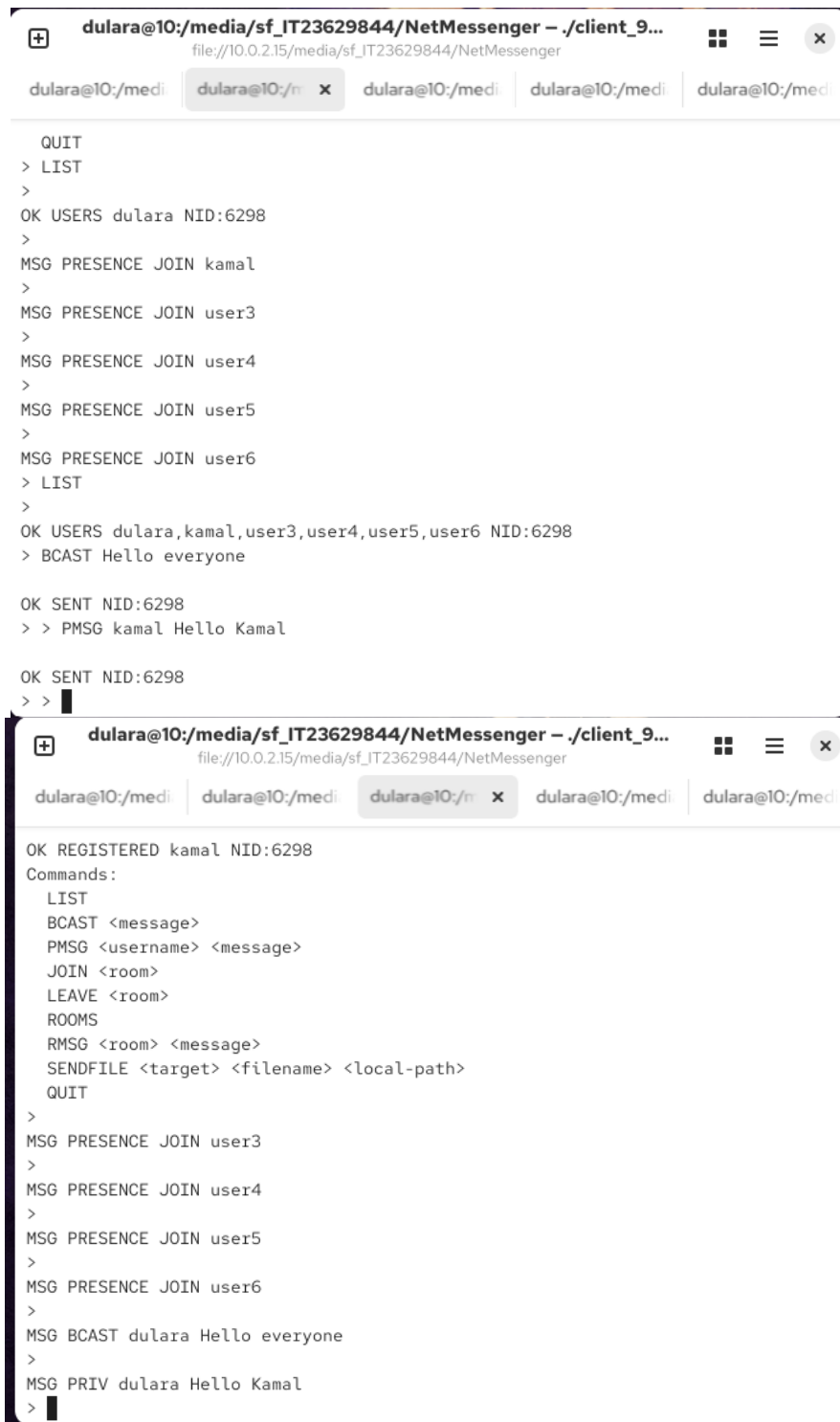
dulara@10:/medi | dulara@10:/n x | dulara@10:/medi | dulara@10:/medi | dulara@10:/medi

ROOMS
RMSG <room> <message>
SENDERFILE <target> <filename> <local-path>
QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone

OK SENT NID:6298
> >
```

6.3 Private Messaging

PMSG identifies a target username and sends the message only to that user. A private message was tested from dulara to kamal and the recipient successfully received it.



The image displays two screenshots of a NetMessenger terminal window. The window title is 'dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...' and the file path is 'file:///10.0.2.15/media/sf_IT23629844/NetMessenger'. The terminal shows a series of commands and responses. In the first screenshot, the user enters 'QUIT', followed by '> LIST', and then a series of 'MSG PRESENCE JOIN' commands for users 'kamal', 'user3', 'user4', 'user5', and 'user6'. After another '> LIST', the response shows 'OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298'. The user then enters '> BCAST Hello everyone', followed by 'OK SENT NID:6298'. Finally, the user enters '> > PMSG kamal Hello Kamal', and the response is 'OK SENT NID:6298'. The second screenshot shows the user entering 'OK REGISTERED kamal NID:6298', followed by a list of commands: 'LIST', 'BCAST <message>', 'PMSG <username> <message>', 'JOIN <room>', 'LEAVE <room>', 'ROOMS', 'RMSG <room> <message>', 'SENDFILE <target> <filename> <local-path>', and 'QUIT'. The user then enters a series of 'MSG PRESENCE JOIN' commands for 'user3', 'user4', 'user5', and 'user6', followed by 'MSG BCAST dulara Hello everyone', and finally 'MSG PRIV dulara Hello Kamal'.

```
QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone

OK SENT NID:6298
> > PMSG kamal Hello Kamal

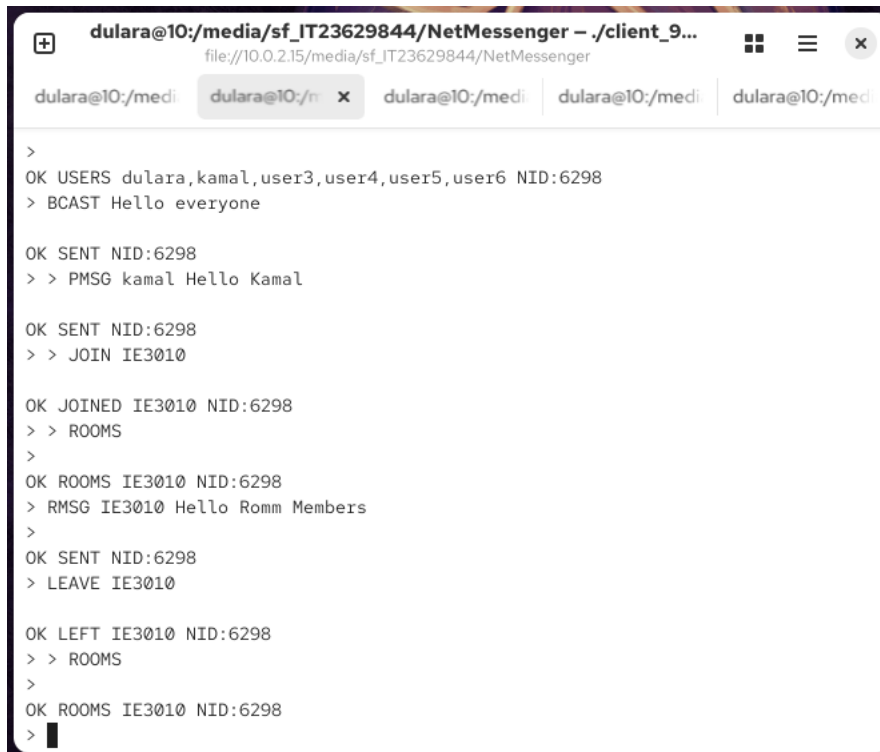
OK SENT NID:6298
> > █

OK REGISTERED kamal NID:6298
Commands:
  LIST
  BCAST <message>
  PMSG <username> <message>
  JOIN <room>
  LEAVE <room>
  ROOMS
  RMSG <room> <message>
  SENDFILE <target> <filename> <local-path>
  QUIT
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
>
MSG BCAST dulara Hello everyone
>
MSG PRIV dulara Hello Kamal
> █
```

6.4 Chat Rooms

JOIN, LEAVE, ROOMS and RMSG provide room functionality. JOIN creates or joins a room, ROOMS lists available rooms, RMSG sends a room message and LEAVE removes the client from the room.

The room operations were tested successfully with connected clients.

A screenshot of a terminal window titled "dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...". The terminal shows a series of commands and responses for a room named "IE3010". The commands include: "> OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298", "> BCAST Hello everyone", "OK SENT NID:6298", "> > PMSG kamal Hello Kamal", "OK SENT NID:6298", "> > JOIN IE3010", "OK JOINED IE3010 NID:6298", "> > ROOMS", "> OK ROOMS IE3010 NID:6298", "> RMSG IE3010 Hello Romm Members", "> OK SENT NID:6298", "> LEAVE IE3010", "OK LEFT IE3010 NID:6298", "> > ROOMS", "> OK ROOMS IE3010 NID:6298", and "> ". The terminal has a dark background with light-colored text. The window has a standard Linux-style title bar with a close button on the right.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi dulara@10:/n x dulara@10:/medi dulara@10:/medi dulara@10:/medi

>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone

OK SENT NID:6298
> > PMSG kamal Hello Kamal

OK SENT NID:6298
> > JOIN IE3010

OK JOINED IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> RMSG IE3010 Hello Romm Members
>
OK SENT NID:6298
> LEAVE IE3010

OK LEFT IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> █
```

7. File Sharing Implementation

SENDFILE identifies a target, filename and file size and is followed by the file data. The server stores the received file under:

`./storage/IT23629844/<sender_username>/<filename>`

A test file named test.txt was created and transferred from dulara to kamal. The recipient successfully received the file and the server-side storage was checked.

```

dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi | dulara@10:/medi | dulara@10:/n x | dulara@10:/medi | dulara@10:/medi

ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
>
MSG BCAST dulara Hello everyone
>
MSG PRIV dulara Hello Kamal
> JOIN IE3010

OK JOINED IE3010 NID:6298
> >
MSG ROOM IE3010 dulara Hello Romm Members
>
[FILE] received from dulara: test.txt (52 bytes) -> received_files/test.txt
> █

```

Figure 5: Successful file transfer

The evidence should demonstrate the sender issuing SENDFILE and the recipient receiving the file.

```

dulara@10:/media/sf_IT23629844/NetMessenger$ find storage/IT23629844 -type f -ls
  33      0 -rwxrwx---   1 root    vboxsf      0 0ct  8 09:08 storage/
IT23629844/.gitkeep
  30      1 -rwxrwx---   1 root    vboxsf     52 0ct  8 10:05 storage/
IT23629844/dulara/test.txt
dulara@10:/media/sf_IT23629844/NetMessenger$

```

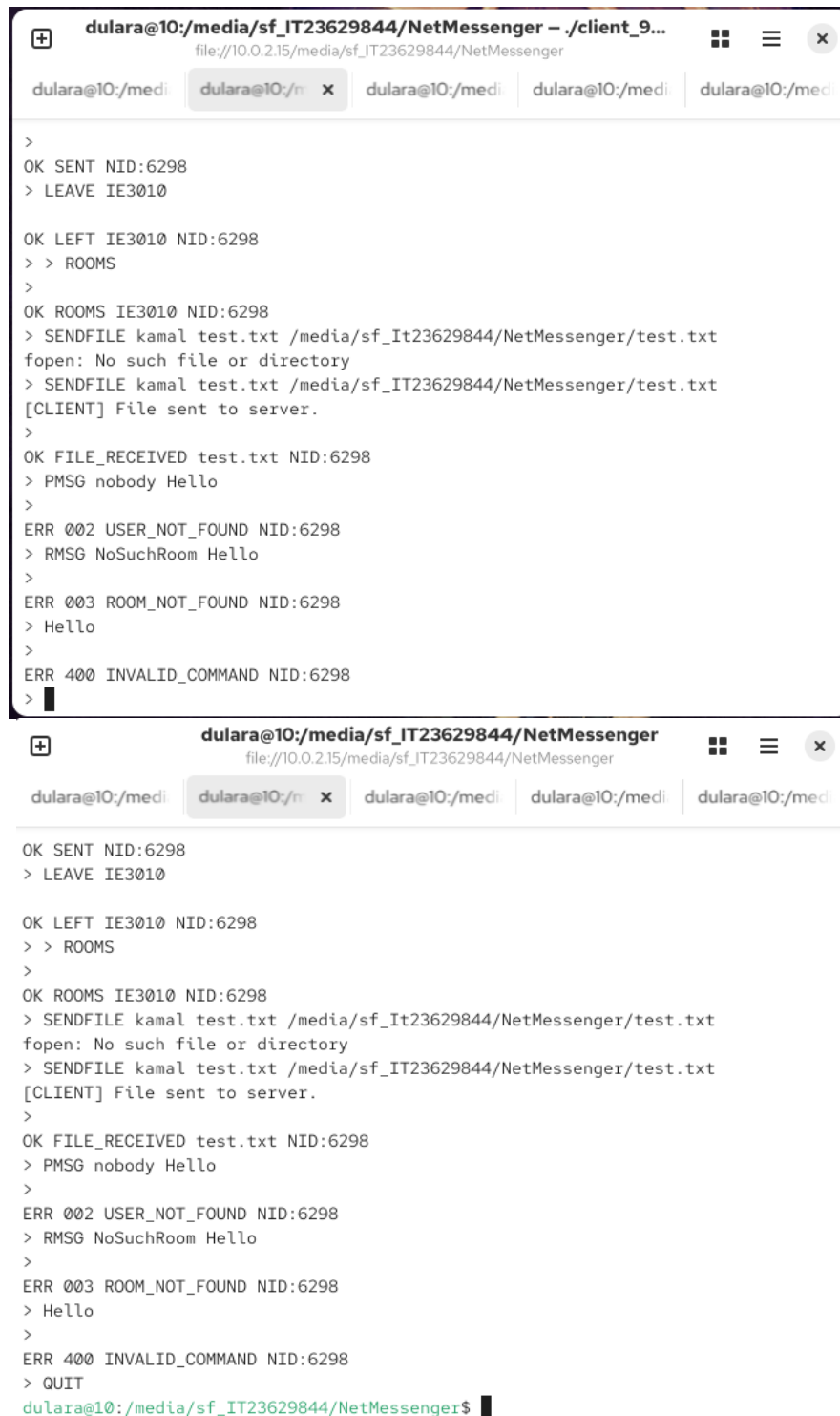
Figure 6: Personalized storage evidence

The storage evidence demonstrates the personalized IT23629844 directory and stored file.

8. Error Handling and Graceful Disconnect

The server is designed to handle invalid commands and invalid targets without terminating the server process. Protocol errors are returned to the requesting client.

QUIT provides graceful termination. The server closes the client connection and removes the associated state. Reconnection was also tested by starting the dulara client again after disconnecting.



The figure consists of two screenshots of a terminal window titled "dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...". The terminal shows a sequence of commands and responses from a client to a server. The commands include: ">", "OK SENT NID:6298", "> LEAVE IE3010", "OK LEFT IE3010 NID:6298", "> > ROOMS", ">", "OK ROOMS IE3010 NID:6298", "> SENDFILE kamal test.txt /media/sf_It23629844/NetMessenger/test.txt", "fopen: No such file or directory", "> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt", "[CLIENT] File sent to server.", ">", "OK FILE_RECEIVED test.txt NID:6298", "> PMSG nobody Hello", ">", "ERR 002 USER_NOT_FOUND NID:6298", "> RMSG NoSuchRoom Hello", ">", "ERR 003 ROOM_NOT_FOUND NID:6298", "> Hello", ">", "ERR 400 INVALID_COMMAND NID:6298", ">". The second screenshot shows the same sequence of commands and responses, but with the final command being "QUIT" instead of ">". The terminal prompt is "dulara@10:/media/sf_IT23629844/NetMessenger\$".

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: x dulara@10:/medi: dulara@10:/medi: dulara@10:/medi:

>
OK SENT NID:6298
> LEAVE IE3010

OK LEFT IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> SENDFILE kamal test.txt /media/sf_It23629844/NetMessenger/test.txt
fopen: No such file or directory
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
[CLIENT] File sent to server.
>
OK FILE_RECEIVED test.txt NID:6298
> PMSG nobody Hello
>
ERR 002 USER_NOT_FOUND NID:6298
> RMSG NoSuchRoom Hello
>
ERR 003 ROOM_NOT_FOUND NID:6298
> Hello
>
ERR 400 INVALID_COMMAND NID:6298
>

dulara@10:/media/sf_IT23629844/NetMessenger

dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: x dulara@10:/medi: dulara@10:/medi: dulara@10:/medi:

OK SENT NID:6298
> LEAVE IE3010

OK LEFT IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> SENDFILE kamal test.txt /media/sf_It23629844/NetMessenger/test.txt
fopen: No such file or directory
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
[CLIENT] File sent to server.
>
OK FILE_RECEIVED test.txt NID:6298
> PMSG nobody Hello
>
ERR 002 USER_NOT_FOUND NID:6298
> RMSG NoSuchRoom Hello
>
ERR 003 ROOM_NOT_FOUND NID:6298
> Hello
>
ERR 400 INVALID_COMMAND NID:6298
> QUIT
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Figure 7: Error handling and graceful disconnect

The evidence should show that an invalid request produces an error response without crashing the server and that a client can disconnect and reconnect.

9. Server-Side Logging

Important runtime events are recorded in the personalized log file:

netmsg_IT23629844.log

The log was checked after the functional tests to confirm that server activity was being recorded.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ tail -n 30 netmsg_IT23629844.log
[2026-10-08 09:16:13] SERVER_START port=15844 nid=NID:6298
[2026-10-08 09:18:56] REGISTER user=dulara
[2026-10-08 09:23:33] REGISTER user=kamal
[2026-10-08 09:25:28] REGISTER user=user3
[2026-10-08 09:26:22] REGISTER user=user4
[2026-10-08 09:27:09] REGISTER user=user5
[2026-10-08 09:28:25] REGISTER user=user6
[2026-10-08 09:41:41] BCAST user=dulara
[2026-10-08 09:48:26] PMSG sender=dulara target=kamal
[2026-10-08 09:53:05] JOIN user=dulara room=IE3010
[2026-10-08 09:53:34] JOIN user=kamal room=IE3010
[2026-10-08 09:53:44] JOIN user=user3 room=IE3010
[2026-10-08 09:54:00] JOIN user=user4 room=IE3010
[2026-10-08 09:54:11] JOIN user=user5 room=IE3010
[2026-10-08 09:54:39] JOIN user=user6 room=IE3010
[2026-10-08 09:55:44] RMSG user=dulara room=IE3010
[2026-10-08 09:58:00] LEAVE user=dulara room=IE3010
[2026-10-08 10:05:15] FILE sender=dulara target=kamal filename=test.txt size=52
[2026-10-08 10:10:44] QUIT user=dulara
[2026-10-08 10:10:44] DISCONNECT user=dulara
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Figure 8: Server-side logging evidence

The log screenshot should show timestamped events generated by the actual NetMessenger execution.

10. Annotated Code Evidence

Socket creation, bind, listen and accept

The server creates a TCP socket, binds it to the personalised port 15844, starts listening for incoming connections, and accepts new client connections for further processing.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -E "socket\(|bind\(|listen\(|accept\(|" server_9844.c
534:     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
546:     if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
549:     if (listen(server_fd, MAX_CLIENTS) < 0) {
564:         int fd = accept(server_fd, (struct sockaddr *)&cliaddr, &len);
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Concurrency/client handling

```
592:         pthread_mutex_lock(&state_lock);
594:         pthread_mutex_unlock(&state_lock);
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n "pthread_create" server_9844.c
590:         if (pthread_create(&c->thread, NULL, client_thread, c) != 0) {
dulara@10:/media/sf_IT23629844/NetMessenger$
```

REGISTER and user state.

The REGISTER handling validates the requested username, checks whether the username is already in use, stores the registered client state, and returns the personalised NID-tagged response.

```

dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -E "REGISTER|USERNAME_TAKEN
|registerd|username" server_9844.c | head -20
41:   char username[MAX_NAME];
161:   strcmp(cclients[i].username, name) == 0) return &clients[i];
204:static void broadcast_presence(const char *kind, const char *username, Clie
nt *exclude) {
214:   snprintf(msg, sizeof(msg), "MSG PRESENCE %s %s", kind, username);
224:   strncat(list, clients[i].username, sizeof(list)-strlen(list)-1);
254:   snprintf(out, sizeof(out), "MSG BCAST %s %s", sender->username, message)
;
260:   snprintf(out, sizeof(out), "MSG PRIV %s %s", sender->username, message);
274:   room->name, sender->username, message);
316:   if (send_file_to_client(user, sender->username, filename, data, size
) < 0)
320:       send_file_to_client(targets[i], sender->username, filename, data
, size);
324:       sender->username, target_name, filename, size);
328:   char username[MAX_NAME] = "";
333:   strncpy(username, c->username, sizeof(username)-1);
343:   log_event("DISCONNECT user=%s", username);
344:   broadcast_presence("LEAVE", username, NULL);
349:static int handle_register(Client *c, const char *username) {
350:   if (!username || !*username || strlen(username) >= MAX_NAME) {
356:   if (find_user_locked(username)) {

dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -A25 -B5 "REGISTER" server_
9844.c | head -35
361-   c->registered = 1;
362-   strncpy(c->username, username, MAX_NAME-1);
363-   c->username[MAX_NAME-1] = '\0';
364-   pthread_mutex_unlock(&state_lock);
365-
366-   send_ok(c, "REGISTERED %s", c->username);
367-   log_event("REGISTER user=%s", c->username);
368-   broadcast_presence("JOIN", c->username, c);
369-   return 1;
370-}
371-
372-static void *client_thread(void *arg) {
373-   Client *c = (Client *)arg;
374-   char line[MAX_LINE];
375-
376-   int r = recv_line(c->fd, line, sizeof(line));
377-   if (r != 1) { cleanup_client(c); return NULL; }
378-
379-   char cmd[32], arg1[MAX_NAME];
380-   if (sscanf(line, "%31s %31s", cmd, arg1) != 2 ||
381-       strcmp(cmd, "REGISTER") != 0) {

```

BCAST and PMSG

The server passes BCAST and PMSG commands and routes broadcast messages to connected clients while private messages are delivered only to the specified user.

```

dulara@10:~$ cd /media/sf_IT23629844/NetMessenger
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n "BCAST" server_9844.c
254:   snprintf(out, sizeof(out), "MSG BCAST %s %s", sender->username, message)
;
400:   } else if (strncmp(line, "BCAST ", 6) == 0) {
405:       log_event("BCAST user=%s", c->username);
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n "PMSG" server_9844.c
406:   } else if (strncmp(line, "PMSG ", 5) == 0) {
419:       log_event("PMSG sender=%s target=%s", c->username, target_na
me);
dulara@10:/media/sf_IT23629844/NetMessenger$ █

```

JOIN, LEAVE, ROOMS and RMSG

The server handles JOIN and LEAVE requests to manage room membership, ROOMS to list available rooms, and RMSG to route messages to members of the selected room.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -E "JOIN|LEAVE|ROOMS|RMSG"
server_9844.c
28:#define MAX_ROOMS 50
54:static Room rooms[MAX_ROOMS];
166:    for (int i = 0; i < MAX_ROOMS; ++i)
174:    for (int i = 0; i < MAX_ROOMS; ++i) {
234:    for (int i = 0; i < MAX_ROOMS; ++i) {
241:    send_ok(c, "ROOMS %s", list);
336:    for (int i = 0; i < MAX_ROOMS; ++i)
344:        broadcast_presence("LEAVE", username, NULL);
368:    broadcast_presence("JOIN", c->username, c);
421:    } else if (strcmp(line, "JOIN ", 5) == 0) {
438:        send_ok(c, "JOINED %s", room_name);
439:        log_event("JOIN user=%s room=%s", c->username, room_name);
441:    } else if (strcmp(line, "LEAVE ", 6) == 0) {
454:        log_event("LEAVE user=%s room=%s", c->username, room_name);
456:    } else if (strcmp(line, "ROOMS") == 0) {
458:    } else if (strcmp(line, "RMSG ", 5) == 0) {
473:        log_event("RMSG user=%s room=%s", c->username, room_name);
dulara@10:/media/sf_IT23629844/NetMessenger$
```

```
dulara@10:/media/sf_IT23629844/NetMessenger$ sed -n '421,478p' server_9844.c
    } else if (strcmp(line, "JOIN ", 5) == 0) {
        char room_name[MAX_NAME];
        if (sscanf(line + 5, "%31s", room_name) != 1) {
            send_err(c, "400 INVALID_ROOM");
            continue;
        }
        pthread_mutex_lock(&state_lock);
        Room *room = get_or_create_room_locked(room_name);
        int joined = 0;
        if (room && !room_has_member(room, c) && room->member_count < MAX_ROOM_MEMBERS) {
            room->members[room->member_count++] = c;
            joined = 1;
        }
        pthread_mutex_unlock(&state_lock);
        if (!room) send_err(c, "005 ROOM_LIMIT");
        else if (!joined) send_err(c, "004 ALREADY_IN_ROOM");
        else {
            send_ok(c, "JOINED %s", room_name);
            log_event("JOIN user=%s room=%s", c->username, room_name);
        }
    } else if (strcmp(line, "LEAVE ", 6) == 0) {
        char room_name[MAX_NAME];
```

```
        if (sscanf(line + 6, "%31s", room_name) != 1) {
            send_err(c, "400 INVALID_ROOM");
            continue;
        }
        pthread_mutex_lock(&state_lock);
        Room *room = find_room_locked(room_name);
        if (room) remove_from_room_locked(room, c);
        pthread_mutex_unlock(&state_lock);
        if (!room) send_err(c, "003 ROOM_NOT_FOUND");
        else {
            send_ok(c, "LEFT %s", room_name);
            log_event("LEAVE user=%s room=%s", c->username, room_name);
        }
    } else if (strcmp(line, "ROOMS") == 0) {
        send_rooms(c);
    } else if (strcmp(line, "RMSG ", 5) == 0) {
        char room_name[MAX_NAME], msg[MAX_LINE];
        if (sscanf(line + 5, "%31s %[^\\n]", room_name, msg) < 2) {
            send_err(c, "400 INVALID_MESSAGE");
            continue;
        }
        pthread_mutex_lock(&state_lock);
        Room *room = find_room_locked(room_name);
        int member = room && room_has_member(room, c);
        pthread_mutex_unlock(&state_lock);
```

```

        room = find_room_locked(room_name);
        int member = room && room_has_member(room, c);
        pthread_mutex_unlock(&state_lock);
        if (!room) send_err(c, "003 ROOM_NOT_FOUND");
        else if (!member) send_err(c, "006 NOT_IN_ROOM");
        else {
            deliver_room(c, room, msg);
            send_ok(c, "SENT");
            log_event("RMSG user=%s room=%s", c->username, room_name);
        }
    } else if (strncmp(line, "SENDFILE ", 9) == 0) {
        char target[MAX_NAME], filename[256];
        unsigned long long filesize = 0;
        char extra[8];
        dulara@10:/media/sf_IT23629844/NetMessenger$ █

```

SENDFILE receive/store/forward

```

dulara@10:/media/sf_IT23629844/NetMessenger$ grep -n -E "SENDFILE|storage|fwrite
|fread|send_file|FILE_RECEIVED" server_9844.c
278:static int send_file_to_client(Client *target, const char *sender,
293:static void send_file_to_target(Client *sender, const char *target_name,
316:    if (send_file_to_client(user, sender->username, filename, data, size
) < 0)
320:        send_file_to_client(targets[i], sender->username, filename, data
, size);
322:    send_ok(sender, "FILE_RECEIVED %s", filename);
475:    } else if (strncmp(line, "SENDFILE ", 9) == 0) {
492:        snprintf(dir, sizeof(dir), "./storage/IT23629844/%s", c->usernam
e);
493:        mkdir("./storage", 0755);
494:        mkdir("./storage/IT23629844", 0755);
503:        fwrite(data, 1, (size_t)filesize, f);
506:        send_file_to_target(c, target, filename, data, (size_t)filesize)
;
531:    mkdir("storage", 0755);
532:    mkdir("storage/IT23629844", 0755);
dulara@10:/media/sf_IT23629844/NetMessenger$ █

```

Validation and ERR response.

The server validates client requests and generates appropriate error responses for invalid or unsupported requests. The personalised NID:6298 tag is included in server responses.

```

dulara@10:/media/sf_IT23629844/NetMessenger$ sed -n '105,120p' server_9844.c
    vsnprintf(body, sizeof(body), fmt, ap);
    va_end(ap);
    snprintf(out, sizeof(out), "OK %s %s", body, NID);
    return send_line(c, out);
}

static int send_err(Client *c, const char *fmt, ...) {
    char body[MAX_LINE];
    char out[MAX_LINE + 64];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(body, sizeof(body), fmt, ap);
    va_end(ap);
    snprintf(out, sizeof(out), "ERR %s %s", body, NID);
    return send_line(c, out);
}

```

```

}
dulara@10:/media/sf_IT23629844/NetMessenger$ sed -n '507,520p' server_9844.c
    free(data);
    } else if (strcmp(line, "QUIT") == 0) {
        send_ok(c, "BYE");
        log_event("QUIT user=%s", c->username);
        break;
    } else {
        send_err(c, "400 INVALID_COMMAND");
    }
}

cleanup_client(c);
return NULL;
}

```

Disconnect cleanup and logging

The server records important events in the personalised log file and performs cleanup when a client disconnects, including closing the connection and removing the client from active server state.

```

dulara@10:/media/sf_IT23629844/NetMessenger$ sed -n '53,68p' server_9844.c
static Client clients[MAX_CLIENTS];
static Room rooms[MAX_ROOMS];
static pthread_mutex_t state_lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t log_lock = PTHREAD_MUTEX_INITIALIZER;
static volatile sig_atomic_t running = 1;

static void log_event(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    pthread_mutex_lock(&log_lock);
    FILE *f = fopen("netmsg_IT23629844.log", "a");
    if (f) {
        time_t now = time(NULL);
        struct tm tmv;
        localtime_r(&now, &tmv);
        fprintf(f, "[%04d-%02d-%02d %02d:%02d:%02d] ",
            ....
    }

dulara@10:/media/sf_IT23629844/NetMessenger$ sed -n '595,615p' server_9844.c
    } else {
        pthread_detach(c->thread);
    }
}

close(server_fd);
log_event("SERVER_STOP");
return 0;
}

```

11. Execution and Output Evidence

Server startup on port 15844

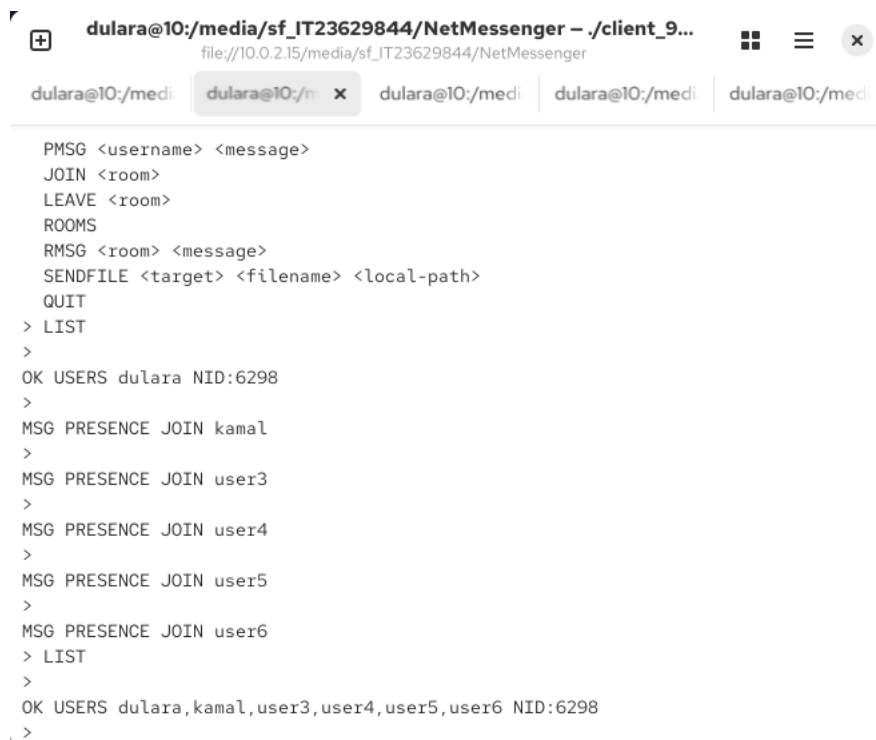
The server was successfully started and displayed the personalised registration details, listening port 15844 and Node ID NID:6298.

TCP listener evidence

The ss command confirms that the NetMessenger server is actively listening for TCP connections on port 15844.

Six simultaneous clients

Six clients (dulara, kamal, user3, user4, user5, and user6) were connected simultaneously, demonstrating the multi-client capability of the server.



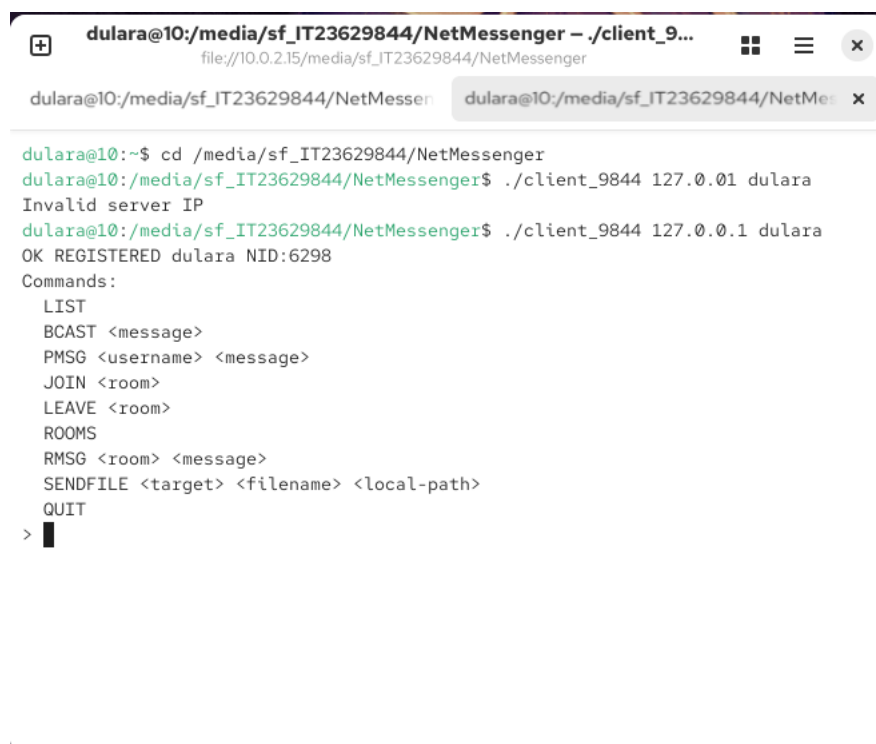
```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi dulara@10:/n x dulara@10:/medi dulara@10:/medi dulara@10:/medi

PMSG <username> <message>
JOIN <room>
LEAVE <room>
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
>
```

Registration/presence

The client dulara was successfully registered with the personalised response NID:6298. JOIN notifications were also received when additional clients connected.



```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/media/sf_IT23629844/NetMessenger dulara@10:/media/sf_IT23629844/NetMe x

dulara@10:~$ cd /media/sf_IT23629844/NetMessenger
dulara@10:/media/sf_IT23629844/NetMessenger$ ./client_9844 127.0.01 dulara
Invalid server IP
dulara@10:/media/sf_IT23629844/NetMessenger$ ./client_9844 127.0.0.1 dulara
OK REGISTERED dulara NID:6298
Commands:
LIST
BCAST <message>
PMSG <username> <message>
JOIN <room>
LEAVE <room>
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
> █
```

LIST

The LIST command returned the currently connected users together with the personalised server response tag.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ ./client_9844 127.0.0.1 dulara
OK REGISTERED dulara NID:6298
Commands:
LIST
BCAST <message>
PMSG <username> <message>
JOIN <room>
LEAVE <room>
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
> LIST
>
OK USERS dulara NID:6298
> █
```

BCAST

A BCAST message was successfully sent from one client and delivered to the other connected clients.



```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger
dulara@10:/medi: dulara@10:/n x dulara@10:/medi: dulara@10:/medi: dulara@10:/medi:
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone
OK SENT NID:6298
> >
```

PMSG

A private message was successfully sent to the specified client using the PMSG command.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: dulara@10:/n x dulara@10:/medi dulara@10:/medi dulara@10:/medi

QUIT
> LIST
>
OK USERS dulara NID:6298
>
MSG PRESENCE JOIN kamal
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
> LIST
>
OK USERS dulara,kamal,user3,user4,user5,user6 NID:6298
> BCAST Hello everyone

OK SENT NID:6298
> > PMSG kamal Hello Kamal

OK SENT NID:6298
> > █
```

Room operations

JOIN, ROOMS, RMSG and LEAVE operations were executed successfully, demonstrating room creation, membership, room listing, room messaging and leaving functionality.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: dulara@10:/medi: dulara@10:/n x dulara@10:/medi: dulara@10:/medi:

JOIN <room>
LEAVE <room>
ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
>
MSG BCAST dulara Hello everyone
>
MSG PRIV dulara Hello Kamal
> JOIN IE3010

OK JOINED IE3010 NID:6298
> >
MSG ROOM IE3010 dulara Hello Romm Members
>
```

SENDFILE

A file was successfully transferred through the server using the SENDFILE command.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file:///10.0.2.15/media/sf_IT23629844/NetMessenger
dulara@10:/medi | dulara@10:/medi | dulara@10:/n x | dulara@10:/medi | dulara@10:/medi

ROOMS
RMSG <room> <message>
SENDFILE <target> <filename> <local-path>
QUIT
>
MSG PRESENCE JOIN user3
>
MSG PRESENCE JOIN user4
>
MSG PRESENCE JOIN user5
>
MSG PRESENCE JOIN user6
>
MSG BCAST dulara Hello everyone
>
MSG PRIV dulara Hello Kamal
> JOIN IE3010

OK JOINED IE3010 NID:6298
> >
MSG ROOM IE3010 dulara Hello Romm Members
>
[FILE] received from dulara: test.txt (52 bytes) -> received_files/test.txt
> █
```

Storage directory

The transferred file was stored under the personalised storage hierarchy

`./storage/IT23629844/<sender_username>./`.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ find storage/IT23629844 -maxdepth 3
-type f -ls
 33      0 -rwxrwx---  1 root    vboxsf      0 Oct  8 09:08 storage/
IT23629844/.gitkeep
 30      1 -rwxrwx---  1 root    vboxsf     52 Oct  8 10:05 storage/
IT23629844/dulara/test.txt
dulara@10:/media/sf_IT23629844/NetMessenger$
```

Error response

An invalid private-message request was handled without terminating the server. The server returned the appropriate `USER_NOT_FOUND` error together with `NID:6298`.

```
dulara@10:/media/sf_IT23629844/NetMessenger - ./client_9...
file://10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: dulara@10:/n x dulara@10:/medi: dulara@10:/medi: dulara@10:/medi:

>
OK SENT NID:6298
> LEAVE IE3010

OK LEFT IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
fopen: No such file or directory
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
[CLIENT] File sent to server.
>
OK FILE_RECEIVED test.txt NID:6298
> PMSG nobody Hello
>
ERR 002 USER_NOT_FOUND NID:6298
> RMSG NoSuchRoom Hello
>
ERR 003 ROOM_NOT_FOUND NID:6298
> Hello
>
ERR 400 INVALID_COMMAND NID:6298
> █
```

QUIT/reconnection

The client terminated its session using QUIT and subsequently reconnected successfully using the same username.

```
dulara@10:/media/sf_IT23629844/NetMessenger
file://10.0.2.15/media/sf_IT23629844/NetMessenger

dulara@10:/medi: dulara@10:/n x dulara@10:/medi: dulara@10:/medi: dulara@10:/medi:

OK SENT NID:6298
> LEAVE IE3010

OK LEFT IE3010 NID:6298
> > ROOMS
>
OK ROOMS IE3010 NID:6298
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
fopen: No such file or directory
> SENDFILE kamal test.txt /media/sf_IT23629844/NetMessenger/test.txt
[CLIENT] File sent to server.
>
OK FILE_RECEIVED test.txt NID:6298
> PMSG nobody Hello
>
ERR 002 USER_NOT_FOUND NID:6298
> RMSG NoSuchRoom Hello
>
ERR 003 ROOM_NOT_FOUND NID:6298
> Hello
>
ERR 400 INVALID_COMMAND NID:6298
> QUIT
dulara@10:/media/sf_IT23629844/NetMessenger$ █
```

Server log

The personalised server log netmsg_IT23629844.log records server-side connection, messaging, file-transfer and disconnection events.

```
dulara@10:/media/sf_IT23629844/NetMessenger$ tail -n 30 netmsg_IT23629844.log
[2026-10-08 09:16:13] SERVER_START port=15844 nid=NID:6298
[2026-10-08 09:18:56] REGISTER user=dulara
[2026-10-08 09:23:33] REGISTER user=kamal
[2026-10-08 09:25:28] REGISTER user=user3
[2026-10-08 09:26:22] REGISTER user=user4
[2026-10-08 09:27:09] REGISTER user=user5
[2026-10-08 09:28:25] REGISTER user=user6
[2026-10-08 09:41:41] BCAST user=dulara
[2026-10-08 09:48:26] PMSG sender=dulara target=kamal
[2026-10-08 09:53:05] JOIN user=dulara room=IE3010
[2026-10-08 09:53:34] JOIN user=kamal room=IE3010
[2026-10-08 09:53:44] JOIN user=user3 room=IE3010
[2026-10-08 09:54:00] JOIN user=user4 room=IE3010
[2026-10-08 09:54:11] JOIN user=user5 room=IE3010
[2026-10-08 09:54:39] JOIN user=user6 room=IE3010
[2026-10-08 09:55:44] RMSG user=dulara room=IE3010
[2026-10-08 09:58:00] LEAVE user=dulara room=IE3010
[2026-10-08 10:05:15] FILE sender=dulara target=kamal filename=test.txt size=52
[2026-10-08 10:10:44] QUIT user=dulara
[2026-10-08 10:10:44] DISCONNECT user=dulara
dulara@10:/media/sf_IT23629844/NetMessenger$
```

12. Testing Summary

The table below records the main tests performed during the development sessions. An automated 19-test regression result is not claimed because such a test suite was not part of the verified project evidence.

Test Case	Expected Result	Actual Result	Status
Unique registration	User registered with NID	dulara registered successfully with NID:6298	PASS
LIST	Connected users returned	Six connected usernames were listed during testing	PASS
Six simultaneous clients	At least five clients served	dulara, kamal, user3, user4, user5 and user6 connected	PASS
BCAST	Other clients receive message	Broadcast successfully delivered	PASS
PMSG	Only target receives message	kamal successfully received dulara's private message	PASS

JOIN / ROOMS / RMSG / LEAVE	Room operations work	Room commands successfully tested	PASS
SENDFILE	Recipient receives file	test.txt successfully received by kamal	PASS
Storage	Server stores copy	Personalized storage checked	PASS
Logging	Events recorded	netmsg_IT23629844.log checked	PASS
QUIT / reconnect	Client disconnects and can reconnect	QUIT and reconnection tested	PASS

13. Design Rationale and Assumptions

TCP was selected because the assignment requires TCP/IP communication and reliable delivery of commands and file data. The server centralizes connected-user state, room membership, message routing and file-transfer processing.

A thread-per-client model was selected because each connected client can be processed independently and the coursework requires a bounded number of simultaneous clients. POSIX pthreads provide a direct way to implement this model.

The supplied protocol is retained rather than replaced with a custom command design. Text commands use newline framing, while SENDFILE uses the declared file size to process the file data.

User and room state is maintained in memory during the server session and is not intended to persist after a server restart. The implementation is an educational coursework system and does not provide production-level authentication, TLS or distributed scalability.

14. Conclusion

The completed NetMessenger implementation provides a TCP-based multi-client communication and file-sharing system personalized for IT23629844. It uses port 15844, NID:6298, personalized source files, server logging and personalized file storage.

The system implements registration, user listing, broadcast messaging, private messaging, presence notifications, chat rooms, file transfer, error handling and graceful disconnection. Testing with six simultaneous clients demonstrated more than the required five-client capacity.

The project provided practical experience with TCP sockets, concurrent programming, client/server state management, protocol implementation, file transfer, Linux development and Git/GitHub.

15. Prompt Log / AI Interaction Record

The following summarizes substantive AI support used during development. The entries describe how the output was evaluated or applied rather than claiming that AI output was accepted without testing.

Date	Tool	Task	Use / Change
03.10.2026	ChatGPT	Assignment requirements and personalization	Calculated port 15844 and NID:6298 and identified required files.
04.10.2026	ChatGPT	CentOS/Linux setup	Used commands and verified outputs in the environment.
04.10.2026	ChatGPT	TCP server/client implementation	Implemented, compiled and tested the client/server.
04.10.2026	ChatGPT	Compilation troubleshooting	Resolved missing pthread include and rebuilt successfully.
05.10.2026	ChatGPT	REGISTER/LIST/multi-client testing	Tested with multiple client instances.
05.10.2026	ChatGPT	BCAST/PMSG testing	Tested broadcast and private messaging.
05.10.2026	ChatGPT	JOIN/LEAVE/ROOMS/RMSG testing	Tested room functionality.
06.10.2026	ChatGPT	SENDFILE/storage testing	Created test.txt,

			transferred it and checked storage.
07.10.2026	ChatGPT	Git/GitHub	Initialized repository, committed and pushed to GitHub.
08.10.2026	ChatGPT	Report preparation	Adapted the report to the actual IT23629844 implementation and evidence.

16. Structured Reflection

During the development of the IE3010 Network Programming assignment, I used ChatGPT as a supporting tool at several stages of the NetMessenger project. I mainly used it to understand the assignment requirements, calculate personalized configuration values, configure the Linux environment, understand TCP socket programming, troubleshoot compilation problems, test client/server communication, organize the project with Git and prepare the final documentation.

AI assistance was useful when I needed to understand unfamiliar Linux commands and socket-programming concepts. It also helped me break large development tasks into smaller steps. I used guidance while testing multiple clients, broadcast messaging, private messaging, chat rooms and file transfer.

However, I learned that AI-generated technical suggestions cannot be accepted without checking them against the actual project. Some suggestions can describe different architecture or assume features that are not present in my implementation. Therefore, I compared suggestions with the assignment specification, my source code and actual terminal output. I also avoided claiming features that were not verified.

One important lesson was understanding TCP as a reliable byte-stream communication mechanism. The project improved my knowledge of socket creation, connection handling, concurrent clients, command processing, message routing, room management and file transfer.

The project also improved my confidence in Linux development and Git/GitHub. I learned that successful compilation is not enough; a network program must be tested with real connections and realistic command sequences. Overall, AI helped me work more efficiently, but the final implementation decisions, compilation, execution and testing remained my responsibility.

17. References

IE3010 Network Programming Assignment 2026 – NetMessenger: A Multi-Client Chat and File-Sharing Platform over TCP/IP.

Linux manual pages for socket, bind, listen, accept, connect, send, recv, close and pthread functions.

GNU Compiler Collection (GCC) documentation.

Git documentation.

GitHub repository: <https://github.com/DMFernando0315/IT23629844-NetMessenger>

18. Appendix

Appendix A – Project Files

server_9844.c

client_9844.c

Makefile_9844

README.md

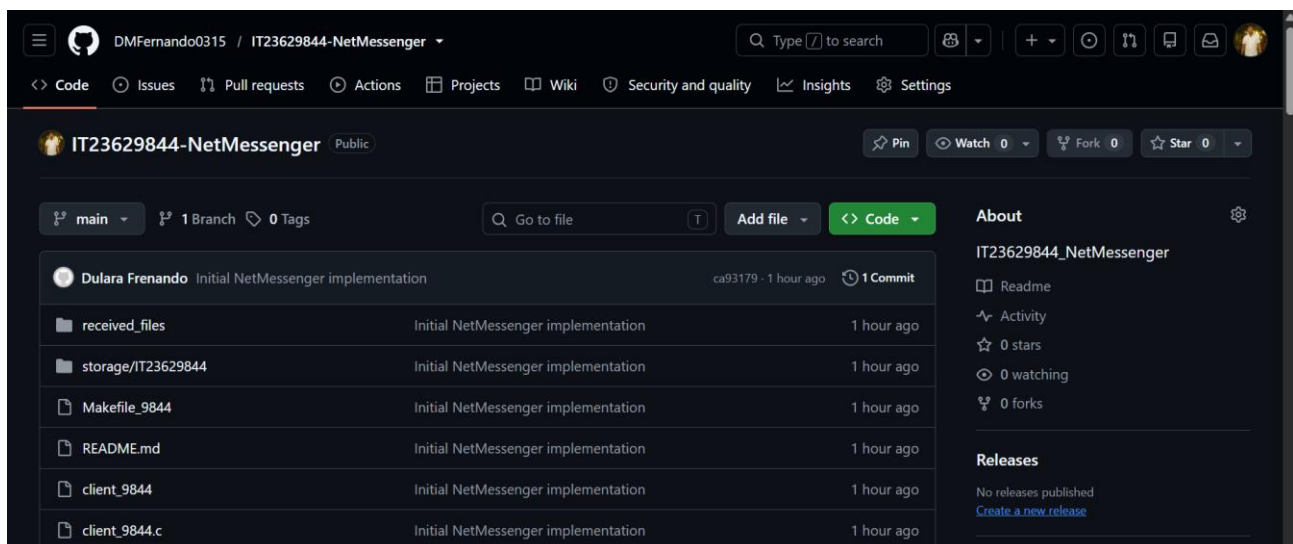
netmsg_IT23629844.log

storage/IT23629844/

received_files/

test.txt

Appendix B – GitHub Repository



<https://github.com/DMFernando0315/IT23629844-NetMessenger>

Appendix C – Evidence Checklist

Server startup

TCP listener

Six clients

Registration/presence

LIST

BCAST

PMSG

Rooms

SENDFILE

Storage

Errors

QUIT/reconnect

Log

Code evidence

IE3010 – IT23629844 – NetMessenger